

# **SECURE TRUST OTA: A RESILIENT AND TRUST-BASED OTA FRAMEWORK FOR IOT DEVICES**

25-26J-058

Project Proposal Report

**P A S N Perera**

Information Technology Specializing in Cyber Security

Department of Computer Systems Engineering

Faculty of Computing

Sri Lanka Institute of Information Technology

September 2025

# **SECURE TRUST OTA: A RESILIENT AND TRUST-BASED OTA FRAMEWORK FOR IOT DEVICES**

25-26J-058

Project Proposal Report

P A S N Perera

Information Technology Specializing in Cyber Security

Department of Computer Systems Engineering

Faculty of Computing

Sri Lanka Institute of Information Technology

September 2025

## **Abstract**

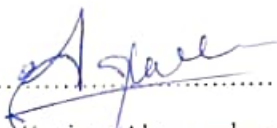
The rapid expansion of the Internet of Things (IoT) has created an urgent requirement for secure and resilient Over-the-Air (OTA) firmware update processes. While OTA updates are needed for the fixing of vulnerabilities and enhancements in functionality, existing frameworks focus mainly on secure transmission and update integrity, neglecting the operational state of devices. Updating devices in abnormal or infected states can magnify failures or propagate malware. This paper describes Secure Trust OTA, a lightweight and trust-based OTA solution that integrates anomaly-aware verification, zero-trust verification, and fail-safe recovery. The basic building block is a Threshold-Based Anomaly Detection Engine, which authenticates device trustworthiness before updates by monitoring telemetry such as CPU, memory, battery, and network activity. Unlike compute-intensive machine learning models, this paper uses light-weight thresholding for real-time detection for resource-limited devices. The method includes secure telemetry collection, centralized threshold administration, anomaly evaluation, and gating of updates. Experimental validation under simulated attack and fault scenarios demonstrates effective anomaly detection with minimal overhead, thereby preventing harmful updates and improving OTA resilience. This work advances IoT security by including trust and anomaly detection in the OTA life cycle itself and contributing to both academic and industrial value.

### Declaration

I declare that this is our work, and this proposal does not incorporate without acknowledgment of any material previously submitted for a degree or diploma in any other university or Institute of higher learning, and to the best of our knowledge and belief, it does not contain any material previously published or written by another person except where the acknowledgment is made in the text. Also, I hereby grant to the Sri Lanka Institute of Information Technology, the nonexclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic, or other mediums. I retain the right to use this content in whole or part in future works (such as articles or books).

| Name           | Student ID | Signature                                                                           |
|----------------|------------|-------------------------------------------------------------------------------------|
| P A S N Perera | IT22307262 |  |

The above candidates are carrying out research for the undergraduate Dissertation under my supervision.

  
.....  
Mr. Kavinga Abeywardena:  
Supervisor

18/09/2025  
.....

Date

.....  
Ms. Dinithi Pandithage  
Co-Supervisor

.....  
Date

## Table of Contents

|                                           |    |
|-------------------------------------------|----|
| Abstract .....                            | 1  |
| Declaration .....                         | 2  |
| List of Figures .....                     | 4  |
| List of Tables .....                      | 4  |
| 1. Introduction .....                     | 5  |
| 2. Background & Literature Survey .....   | 8  |
| 3. Research Gap .....                     | 10 |
| 4. Research Problem .....                 | 11 |
| 5. Objectives .....                       | 12 |
| 5.1. Main Objective .....                 | 12 |
| 5.2. Specific Objectives .....            | 12 |
| 6. Methodology .....                      | 13 |
| 6.1. System Architecture .....            | 13 |
| 6.2. Model Architecture .....             | 14 |
| 7. Project Timeline .....                 | 16 |
| 8. Project Requirements .....             | 16 |
| 8.1. Functional Requirements: .....       | 16 |
| 8.2. Non-Functional Requirements: .....   | 17 |
| 8.3. System Requirements .....            | 17 |
| 8.4. User Requirements .....              | 18 |
| 9. Test Cases .....                       | 18 |
| 10. Budget and budget Justification ..... | 20 |
| 10.1. Budget .....                        | 20 |
| 10.2. Budget Justification .....          | 22 |
| References .....                          | 23 |

## List of Figures

|                                     |    |
|-------------------------------------|----|
| Figure 1: System Architecture ..... | 13 |
| Figure 2: Model Architecture.....   | 14 |
| Figure 3: Project Timeline .....    | 16 |

## List of Tables

|                                                                  |    |
|------------------------------------------------------------------|----|
| Table 1: Comparison of Related Work with Proposed Solution ..... | 9  |
| Table 2: Budget and Justification.....                           | 20 |

# 1. Introduction

The rapid expansion of the Internet of Things (IoT) ecosystem has witnessed billions of connected devices being deployed in critical sectors such as healthcare, transportation, manufacturing, and smart cities [1] [2]. The devices tend to operate autonomously and round the clock and therefore must have efficient and secure mechanisms for updating their firmware and software integrity [3]. Over-the-Air (OTA) update mechanisms have emerged as the preferred means of remotely installing firmware patches, bug fixes, and security updates in IoT devices without physical access [4] [5]. OTA updates are not only necessary for improving functionality but also for patching newly discovered vulnerabilities that could otherwise be exploited by attackers.

Despite their significance, existing OTA update systems still face several challenges [6]. Traditional OTA mechanisms may not have robust trust verification, anomaly detection, or resilience against adversarial attacks such as Distributed Denial-of-Service (DDoS), ransomware, or firmware manipulation [7]. Furthermore, IoT devices with limited resources typically cannot support heavyweight security mechanisms, thus lightweight yet secure solutions are imperative. [8] Current solutions mainly focus on secure transmission (e.g., TLS, HTTPS) and update file integrity checks but do not typically support pre-update anomaly detection, zero-trust authorization, and fail-safe recovery in a systematic and integrated manner [9]. This leaves a significant knowledge gap in delivering OTA solutions that are resilient, trustworthy, and adaptive to anomalous behavior in IoT device operation.

To bridge this gap, this research introduces Secure Trust OTA: A Resilient and Trust-Based OTA Framework for IoT Devices, which integrates four custom-developed components: (1) a lightweight secure OTA core, (2) a zero-trust authorization layer, (3) a threshold-based anomaly detection engine, and (4) a resilient fail-safe update engine. Among them, the Threshold-Based Anomaly Detection Engine is a primary facilitator of trustworthiness since it monitors IoT device telemetry data in real time before updating them. This proactive detection measure not only prevents infected updates from spreading but also ensures that updates are only received by devices that are in a stable and trusted state.

The novelty of this research lies in the design and integration of a non-machine learning, threshold-based anomaly detection mechanism directly into the IoT Over-the-Air (OTA) update process. Existing OTA frameworks such as Mender [10] and Eclipse Hawkbit [11] primarily focus on the secure transfer and authentication of firmware but rely heavily on rollback mechanisms or manual interventions when failures occur, offering reactive recovery rather than proactive prevention. Similarly, academic studies emphasize that firmware updates are among the most exploited attack vectors in IoT ecosystems [12] [13], yet little attention has been given to proactively preventing device instability before updates begin.

Current anomaly detection solutions in IoT largely adopt machine learning [14] [15] or deep learning models [16], which, although effective, are computationally and memory intensive, making them unsuitable for resource-constrained IoT devices deployed in critical infrastructures such as healthcare, smart grids, and industrial systems [17]. In contrast, threshold-based mechanisms have proven effective in lightweight anomaly detection for energy monitoring, industrial IoT, and device health telemetry. However, their direct application in securing OTA update reliability remains unexplored.

This research introduces a lightweight, rule-driven anomaly detection engine that continuously monitors device telemetry (CPU usage, memory, disk space, and network stability) and compares values against predefined adaptive thresholds. By ensuring that updates are only attempted under stable operating conditions, the framework shifts the paradigm from reactive rollback to proactive prevention. This not only reduces the risk of failed updates and device bricking but also improves overall trust in OTA frameworks, aligning with broader calls for resilient IoT security architectures. The suggested research lies in the area of IoT system security, leveraging the foundations of data science and anomaly detection for device behavior monitoring, zero-trust security for authorization, and resilient system design for recovery. In embracing these pillars, the framework not only assures firmware authenticity and confidentiality but also actively mitigates risks arising from device anomalies or adversarial interference.

From a practical perspective, the Secure Trust OTA system demonstrates outstanding commercialization potential as secure and resilient OTA updating is a pressing demand for companies deploying large-scale IoT infrastructures.

IoT devices are fast becoming the foundation for mission-critical operations in contemporary businesses in various sectors like healthcare, smart cities, industrial automation, and logistics. Any glitch in firmware reliability or integrity of OTA operations can result in massive financial losses, operational downtime, and loss of brand reputation. Therefore, products that offer secure, reliable, and anomaly-aware OTA updates are extremely popular.

**Healthcare IoT:** Medical devices, such as infusion pumps, patient monitors, and wearable health sensors, are utilized by hospitals. Firmware vulnerabilities within these devices can be used to modify patient information or even disrupt life-critical care. Secure Trust OTA guards against updating the devices securely, preventing the propagation of bugs or malware updates and enhancing patient safety.

**Industrial IoT (IIoT):** Sensor networks, robotic arms, and PLC systems are all relied upon by manufacturing factories to automate. Production lines can be brought to a complete standstill when faulty firmware updates initiate downtime, resulting in millions of losses per hour. The threshold-based anomaly detection engine is utilized by the system to make sure that only devices that are running safely are updated, reducing downtime and protecting continuity of operations.



Smart Utilities and Cities: Distributed IoT networks enable smart grids, traffic management systems, and water treatment plants. Bulk device updates without pre-update authentication can lead to network-wide failures or cascading system crashes. Secure Trust OTA uses centralized monitoring, adaptive thresholding, and fail-safe that ensures critical infrastructure is not brought down in the middle of bulk updates.

Consumer IoT: Manufacturers of smart home devices (cams, locks, thermostats) stand to lose reputation if firmware updates fail or get hacked. Secure Trust OTA offers lean monitoring suitable even for low-resource consumer devices so companies can deliver reliable updates without full-system revamps.

In brief, Secure Trust OTA addresses a central gap in IoT security with significant commercial applicability across numerous industries. Its combination of anomaly-aware pre-update validation, zero-trust authorization, and fail-safe functionality not only improves the security of devices and operational performance but also represents a genuine entrepreneurial opportunity for IoT cybersecurity businesses and product makers looking to differentiate themselves in a saturated marketplace.

## 2. Background & Literature Survey

The Internet of Things (IoT) has revolutionized several industries by enabling connected devices to collect and exchange information [1] [2]. As IoT devices have rapidly increased, new primary security concerns, particularly pertaining to the deployment of Over-the-Air (OTA) firmware updates, have emerged. Updates are required to maintain device functionality, remove vulnerabilities, and introduce new features [10] [3]. Although critical, OTA processes are susceptible to attacks such as unauthorized firmware updates, downgrade attacks, and denial-of-service [5] [12] [13]. Safe and effective OTA processes are necessary to prevent these attacks and maintain the integrity and reliability of IoT systems.

Firmware updates are crucial for maintaining and enhancing IoT devices. Traditional firmware update methods involve physical access to devices, which is impractical for mass deployment. OTA updates address this limitation by enabling remote updating over wireless networks. However, the OTA procedure introduces new security concerns. Malicious actors can exploit vulnerabilities in the update process to compromise devices, potentially causing data breaches, service disruptions, or unauthorized device control.

Various security mechanisms have been proposed to counter these threats, including cryptographic techniques such as encryption and digital signatures to authenticate and protect firmware integrity. Secure boot mechanisms and rollback protection schemes have also been integrated to prevent execution of unauthorized firmware. Despite these advancements, protecting OTA updates, especially in resource-constrained IoT devices, remains challenging.

IoT anomaly detection has become increasingly important with the rise in networked devices and security threats. IoT devices generate real-time telemetry such as CPU usage, memory consumption, battery life, and network traffic, from which abnormal patterns indicating potential compromise or failure can be detected. Early approaches used statistical thresholding, where measurements exceeding predetermined limits were classified as anomalies. While machine learning (ML) algorithms have gained popularity in anomaly detection, threshold-based approaches remain relevant due to their low computational cost and suitability for constrained systems.

Research on IoT anomaly detection has focused on intrusion detection, fault tolerance, and network traffic monitoring [18]. Fatty et al highlights that threshold-based schemes efficiently detect denial-of-service attacks and anomalous traffic without sophisticated models [19]. Research demonstrated that metrics like memory leaks, CPU spikes, or unaccounted reboot rates effectively detect device-level anomalies with minimal processing costs. However, these works primarily focus on security monitoring rather than pre-update verification.

OTA updates themselves can become attack vectors if device state anomalies are unmonitored prior to updating. Some show that compromised devices can propagate malware during firmware updates. Existing OTA solutions like Mender [10], Eclipse Hawkbit [11], and Balena [19] provide

reliable distribution and rollback but lack integrated anomaly detection engines that evaluate device trustworthiness before updates . Cloud-based platforms like AWS IoT Device Management and Azure IoT Update Service offer centralized monitoring but mainly support post-failure troubleshooting rather than preventing updates on anomalous devices.

Several studies have attempted to integrate OTA processes with lightweight trust solutions. Somya et al proposed a trust evaluation layer for IoT devices [21], some examined threshold-based monitoring for fault detection in industrial IoT. Some highlighted device health monitoring via telemetry [22] but did not connect anomaly detection to OTA update decision-making. Also, some studies stressed that updating unstable devices increases failures and downtime. However, few studies integrate threshold-based anomaly detection directly as a gatekeeping mechanism within OTA processes.

Threshold-based anomaly detection aligns with zero-trust security principles, which reject implicit trust and require continuous validation of device behavior before granting access to sensitive procedures. While zero-trust has been extensively studied in network security, its application to OTA updates remains underexplored. Most existing anomaly detection research relies on complex ML models such as clustering, autoencoders, or neural networks, which are unsuitable for constrained IoT devices, or focuses on post-incident analysis rather than proactive prevention.

In summary, although literature highlights the importance of anomaly detection in IoT and OTA updates, it exposes a gap in pre-update anomaly detection integration. Existing solutions optimize update distribution and rollback reliability but fail to incorporate proactive device health screening. The proposed Threshold-Based Anomaly Detection Engine addresses this gap by employing lightweight threshold-based strategies coordinated with IoT resource constraints and zero-trust principles, offering a low-overhead, real-time mechanism for enhancing OTA security and reliability.

*Table 1: Comparison of Related Work with Proposed Solution*

| <b>Existing Research</b>                            | <b>Limitations</b>                                                           | <b>Proposed Solution</b>                                                          |
|-----------------------------------------------------|------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Secure OTA delivery, rollback recovery              | Assumes device is stable before updates, no anomaly checks                   | Adds pre-update anomaly detection to ensure updates only happen on stable devices |
| Modular OTA update management, device fleet control | Focus on update orchestration, no health verification before updates         | Integrates health verification mechanism into OTA workflow                        |
| High accuracy in intrusion/fault detection          | High computational overhead, unsuitable for resource-constrained IoT devices | Uses lightweight threshold-based methods ideal for IoT                            |

|                                                                         |                                                                |                                                                 |
|-------------------------------------------------------------------------|----------------------------------------------------------------|-----------------------------------------------------------------|
| Efficient and simple, effective for detecting DoS, CPU/memory anomalies | Typically used for general monitoring, not OTA integration     | Embeds threshold anomaly detection directly into OTA process    |
| Provides anomaly detection and policy enforcement                       | Focuses on general IoT security, not update-specific workflows | OTA-centric anomaly detection ensures update safety             |
| Device and network anomaly detection                                    | Not OTA-focused, no link between anomaly detection and updates | Tight coupling of anomaly detection with update decision-making |

### 3. Research Gap

Despite extensive research on IoT anomaly detection, its integration into OTA update systems is limited. Current OTA platforms like Eclipse Hawkbit [11], Mender [10], and Balena [19] focus on secure firmware delivery, rollback recovery, and integrity verification, assuming that devices are in a trusted and stable state prior to updates. Devices often face anomalous conditions such as CPU or memory spikes, unusual network traffic, log inconsistencies, or degraded sensor input caused by malware, hardware faults, or misconfiguration. Applying updates under such conditions may exacerbate vulnerabilities or permanently damage devices.

Traditional ML-based anomaly detection approaches require significant computational resources and large datasets, limiting their suitability for resource-constrained IoT devices. Threshold-based anomaly detection offers lightweight, interpretable, and real-time solutions, but has rarely been integrated into OTA workflows for pre-update validation. Studies demonstrate threshold-based anomaly detection in smart grids and industrial systems, but not in OTA contexts. Similarly, trust evaluation frameworks like Trust-IoT [20] and cloud monitoring services like AWS IoT Device Defender [21] and Azure IoT Security Center [22] monitor device telemetry but do not prevent updates on anomalous devices.

Dynamic threshold management is also underexplored; most systems use static thresholds, which can lead to false positives or negatives in dynamic IoT environments. Centrally managed, adaptive thresholds compared with real-time telemetry offer a versatile, reliable solution for pre-update validation, but remain largely untested in OTA systems.

Hence, the research gap lies in the lack of lightweight, threshold-based anomaly detection integrated into OTA systems, which can verify device health before updates. The proposed Threshold-Based Anomaly Detection Engine fills this gap, offering a real-time, low-overhead solution that enhances OTA security and resilience without overburdening resource-constrained devices.

## 4. Research Problem

Over-the-Air (OTA) update frameworks for IoT devices are mainly focused on secure transport and integrity verification of updates through mechanisms such as digital signatures, cryptographic hashes, and rollback recovery. These defenses authenticate the authenticity of the update package itself but exclude security and operational state of the device to be updated. In real scenarios, IoT devices usually operate in anomalous states because of malware infections, resource exhaustion, misconfiguration, or sensor faults. Updating devices in such anomalous states may aggravate vulnerabilities, propagate system failures, or even lead to device damage that cannot be reversed. Though anomaly detection has received significant attention in IoT security research, it is most typically machine learning-based, which demands intense computational resources, extensive training data, and is not feasible for computationally limited IoT devices. In contrast, lightweight threshold-based anomaly detection methods, more suitable for constrained settings, have not been systematically applied to OTA procedures. This raises a crucial research problem: there is no threshold-based, light-weight solution integrated in OTA frameworks to authenticate device trustiness before opening updates, and IoT ecosystems are thus exposed to the risks involved in updating unstable or compromised devices.

## **5. Objectives**

### **5.1. Main Objective**

To develop a secure, trustworthy, and resilient Over-The-Air (OTA) firmware update framework for IoT devices that guarantee updates are only installed in devices operating in a stable and trusted environment, thus enhancing the security, reliability, and operational integrity of IoT ecosystems.

### **5.2. Specific Objectives**

To design and implement a lightweight, threshold-based anomaly detection engine that validates IoT device trustworthiness using telemetry data before applying OTA updates.

- To collect and forward device telemetry logs (e.g., CPU usage, memory utilization, connectivity status, error rates) to a central OTA server.
- To define and store baseline threshold values in the central server for critical operational parameters of IoT devices.
- To implement a real-time anomaly detection process that compares incoming telemetry data against predefined thresholds before OTA updates.
- To generate alerts and notifications when anomalies are detected and integrate them into the monitoring dashboard.
- To ensure the anomaly detection mechanism is lightweight, resource-efficient, and suitable for constrained IoT devices.
- To evaluate the effectiveness of the detection engine through testing under both normal and anomalous device conditions.

## 6. Methodology

### 6.1. System Architecture

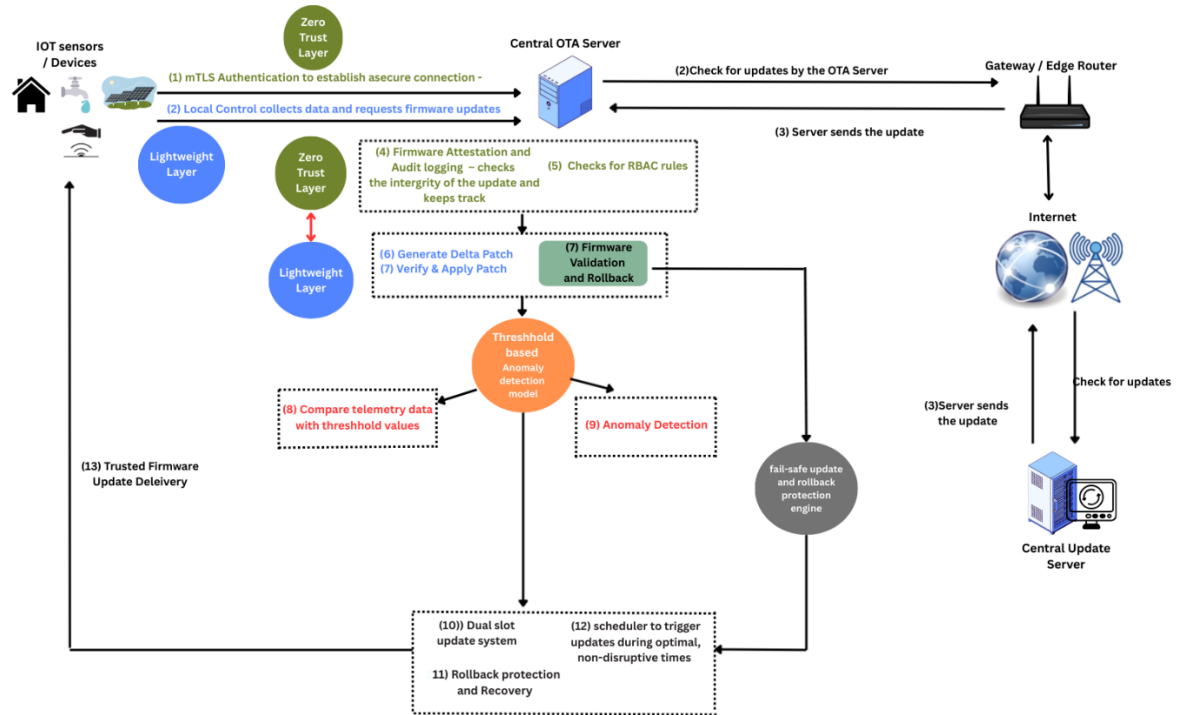


Figure 1: System Architecture

The processes begin with the IoT devices and sensors, which connect through a Zero Trust Layer. The first step in this connection is mutual TLS authentication (mTLS), which establishes a secure communication channel with the OTA server. Once this secure connection is set up, the local control unit of the IoT system collects device data and sends requests for firmware updates.

The OTA server then checks with a central update server to see if any new firmware is available. If updates exist, they are sent back through the OTA server and the gateway or edge router to reach the IoT system. From here, the secure update process unfolds in multiple stages to ensure both safety and reliability.

First, the update undergoes firmware attestation and audit logging, which verifies the integrity of the update and maintains records for accountability. The Zero Trust Layer enforces role-based access control (RBAC) rules to ensure that only authorized actions are allowed. Then, a delta patch is generated rather than replacing the entire firmware, which makes the process lightweight and efficient. The system proceeds to verify and apply the patch, but before it becomes active, firmware validation and rollback mechanisms are in place so that if something goes wrong, the system can return to a previous stable state.

Alongside this, a threshold-based anomaly detection model continuously compares telemetry data from the IoT devices against predefined threshold values. If deviations are found, they are flagged as anomalies and sent to the dashboards for monitoring purposes. If not, the update is being sent to the device.

This anomaly detection feeds into a fail-safe update and rollback protection engine, which ensures that abnormal or malicious updates cannot compromise the device. This engine also enables rollback protection and recovery, dual-slot update systems (so one slot can hold a stable version while the other receives the new patch), and scheduling updates during non-disruptive times to minimize downtime.

## 6.2. Model Architecture

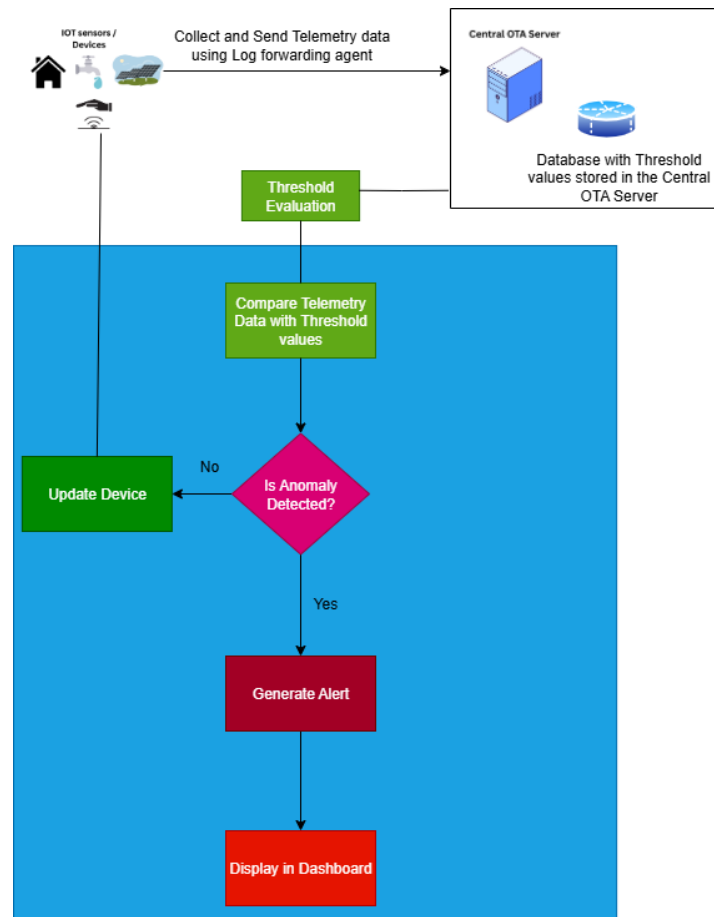


Figure 2: Model Architecture

Threshold-Based Anomaly Detection Engine approach consists of designing, implementing, and testing a lightweight threshold-based system that monitors the operational health and trustworthy behavior of IoT devices before the deployment of Over-the-Air (OTA) updates.

The process begins with telemetry collection, where each IoT device sends critical operation metrics, such as CPU consumption, memory consumption, battery charge level, free storage,



network delay, and error counters, to a server through a secure, authenticated log forwarding agent. The telemetry data are signed to deliver integrity and non-repudiation and protect against tampering or spoofing during transit.

Once telemetry reaches the central OTA server, the pre-update validation module validates against centrally stored threshold values for each device type or group in incoming data. Thresholds are established in advance based on device specification, historical operational baselines, and safety margins, and may also be established to update dynamically with the application of rolling window statistics or percentile-based bounds to provide for reasonable variation.

If no anomalies are detected and if all values are within threshold values, the update is being sent to the devices. Wherever anomalies are detected, the system generates alerts that are directed to an operator dashboard with data about the device, the parameters that exceeded threshold values, and the degree of deviation. Devices that behave abnormally are stopped from receiving OTA updates, thereby blocking the potential unstable or compromised devices from being updated. This type of fail-safe gate mechanism stops unsafe and unverified parameter-operating devices from being updated, following zero-trust security principles.

To validate the effectiveness of the suggested approach, simulation and testing is performed. Normal and faulty telemetry traces are produced, device failure simulated, resource exhaustion simulated, network anomaly simulated, and attacker-controlled devices simulated. True positive rate (TPR), false positive rate (FPR), detection latency, and consumption of resources are measured for evaluating the performance of the engine. Realistic test scenarios include emulated OTA rollouts, concurrent updates of large numbers of devices, and dynamic threshold configuration to simulate dynamic usage scenarios. With this method, the Threshold-Based Anomaly Detection Engine presents a robust, interpretable, and effective solution for improving OTA update security and reliability in IoT systems.

## 7. Project Timeline

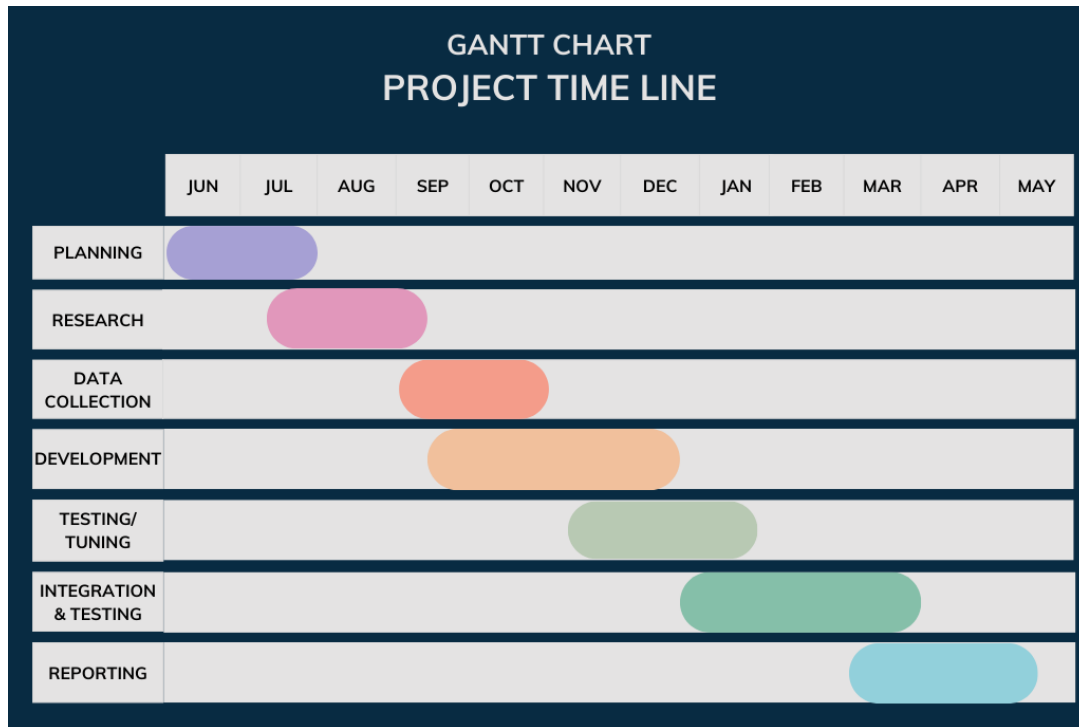


Figure 3: Project Timeline

## 8. Project Requirements

### 8.1. Functional Requirements:

- **Telemetry Collection:** The system must collect operational metrics from IoT devices, including CPU usage, memory utilization, battery level, storage availability, network latency, and error counts.
- **Secure Log Forwarding:** Telemetry data must be sent through a secure, authenticated log forwarding agent to ensure integrity and non-repudiation.
- **Threshold Evaluation:** The system must compare incoming telemetry data against predefined thresholds stored centrally for each device type or group.
- **Adaptive Thresholding:** The system must allow thresholds to be updated dynamically based on rolling statistics or percentile-based bounds.
- **Anomaly Detection:** The system must detect deviations from thresholds, including persistent or correlated anomalies, and generate appropriate alerts.

- **OTA Update Gating:** Devices flagged as anomalous must be blocked from receiving OTA updates until the anomaly is resolved.
- **Alert Generation and Dashboard Integration:** The system must generate alerts with detailed information about the anomaly and forward them to the monitoring dashboard.
- **Debounce Logic & Quorum Rules:** The system must implement rules to prevent false positives from transient anomalies.
- **Audit Logging:** All anomaly detections, alerts, and OTA update decisions must be logged for audit and trace

## 8.2. Non-Functional Requirements:

- **Performance:** The anomaly detection engine must process telemetry data in real-time with minimal latency(<100ms), even for thousands of devices.
- **Scalability:** The system must be scaled to handle large fleets of IoT devices without degradation in performance. Support at least **10,000 devices simultaneously** without >10% degradation in latency.
- **Reliability:** The system must maintain continuous operation (99.9% uptime) and accurately detect anomalies under various operational conditions.
- **Security:** All telemetry data must be transmitted securely using encryption and signature verification.
- **Maintainability:** Threshold rules and monitoring configurations must be easily modifiable by administrators. Modify thresholds in <5 minutes without system downtime
- **Usability:** Alerts and dashboard information must be clear, interpretable, and actionable for operators. Must interpret alerts in <30 seconds and take corrective action with >90% accuracy.
- **Resource Efficiency:** The system must minimize computational and memory usage to remain lightweight and suitable for central servers in constrained environments. CPU utilization should remain under 20% and memory usage under 200 MB for 10,000 devices monitored

## 8.3. System Requirements

- **Hardware:** Central server with moderate CPU (2–4 cores), RAM  $\geq$  8GB, and storage  $\geq$  100GB for telemetry logs. IoT devices must support lightweight telemetry reporting.
- **Software:** Server-side components implemented in Python/Java or similar; secure communication protocols (TLS/HTTPS/MQTT with TLS); time-series database (InfluxDB, TimescaleDB, or equivalent) for telemetry storage.

- **Networking:** Reliable internet or LAN connectivity between devices and central server; support for secure log forwarding agents.
- **Security:** PKI infrastructure for device identity verification, message signing, and certificate management.

## 8.4. User Requirements

- **Device Monitoring:** Users must be able to view the operational status of devices and identify anomalies via a central dashboard.
- **Alert Notifications:** Users must receive timely notifications of detected anomalies, including device details, metric deviations, and suggested actions.
- **OTA Control:** Users must be able to approve or override OTA updates for specific devices while maintaining audit trails.
- **Configurability:** Users must be able to define and update threshold values for different device types without modifying source code.
- **Traceability:** Users must have access to historical telemetry, anomaly detection logs, and update decisions for auditing and analysis.
- **Reporting:** The system must provide periodic reports on detected anomalies, update success rates, and overall device health

## 9. Test Cases

- **Test Case:** Device telemetry within thresholds (CPU, memory, battery, network).  
**Expected Result:** No anomaly detected, OTA update allowed, no alert generated.
- **Test Case:** Device CPU usage exceeds threshold.  
**Expected Result:** Anomaly detected, OTA update blocked, alert sent to dashboard.
- **Test Case:** Device memory usage below safe threshold (possible malfunction).  
**Expected Result:** Anomaly detected, OTA update paused, alert generated and logged.
- **Test Case:** Battery level critically low during update attempt.  
**Expected Result:** Update denied, alert generated, recommendation shown on dashboard.
- **Test Case:** Network packet loss higher than threshold during update.  
**Expected Result:** Update delayed or aborted, anomaly alert generated.
- **Test Case:** Sudden surge in network traffic indicating possible attack.

**Expected Result:** Anomaly flagged, OTA update blocked, alert raised with high severity.

- **Test Case:** Telemetry logs show device error codes outside normal range.  
**Expected Result:** Anomaly detected, update blocked, alert + error details logged.
- **Test Case:** Threshold values dynamically adjusted, and telemetry re-evaluated.  
**Expected Result:** System applies new thresholds, anomaly detection recalibrated, consistent monitoring continues.
- **Test Case:** Database unavailable during anomaly check.  
**Expected Result:** System fails gracefully, OTA update paused, error logged, dashboard notified.
- **Test Case:** Multiple anomalies detected at the same time (CPU + Network).  
**Expected Result:** Both anomalies flagged, update denied, multiple alerts sent with severity prioritization.
- **Test Case:** False-positive check (legitimate traffic spike within tolerance).  
**Expected Result:** No anomaly triggered, update continues, dashboard shows “normal”.
- **Test Case:** Device logs missing or corrupted.  
**Expected Result:** System flags as anomaly, update blocked, alert generated with reason “insufficient data”.
- **Test Case:** OTA update request received while previous anomaly unresolved.  
**Expected Result:** Update denied, dashboard shows pending anomaly resolution required.
- **Test Case:** Alert sent to dashboard.  
**Expected Result:** Dashboard displays correct device ID, anomaly type, timestamp, and severity.
- **Test Case:** Logging of anomaly event.  
**Expected Result:** Anomaly stored in database with full telemetry context for auditing.

## 10. Budget and budget Justification

### 10.1. Budget

Table 2: Budget and Justification

| Item                                                         | Quantity | Unit Cost (LKR) | Total Cost (LKR) | Justification                                                                   |
|--------------------------------------------------------------|----------|-----------------|------------------|---------------------------------------------------------------------------------|
| Laptop/PC (Already existing)                                 | -        | -               | 0                | Already available personal resources.                                           |
| IoT Development Boards (e.g., ESP32 / STM32)                 | 4        | 6,500           | 26,000           | ESP32 boards are cost-effective and widely used in IoT security research.       |
| Secure Boot/Encryption Modules (e.g., ATECC608A Crypto chip) | 2        | 3,500           | 7,000            | Used to simulate hardwarebased cryptographic security and support ECC features. |
| Storage Devices (USB drives / SD cards)                      | 4        | 1,000           | 4,000            | For transfer firmware, test data storage, and recovery testing.                 |
| Networking Devices (Wi-Fi Router / Switch)                   | 1        | 8,500           | 8,500            | Required to simulate enterprise-like IoT device connectivity for OTA delivery.  |

|                                       |   |   |       |                                                                                                            |
|---------------------------------------|---|---|-------|------------------------------------------------------------------------------------------------------------|
| Software Tools & Libraries            | - | - | 0     | Most of the development tools (Arduino IDE,                                                                |
|                                       |   |   |       | PlatformIO, OpenSSL, Wireshark, etc.) are open source.                                                     |
| Cloud Resources (Testing – Free Tier) | - | - | 0     | Cloud services such as AWS IoT Core / Azure IoT Hub will be used within free student credits or free tier. |
| Printing & Documentation              | - | - | 3,000 | Printing of project proposal, final report, and Other essential materials.                                 |
| Cables, Connectors, Power supplies    | - | - | 5,000 | To power-up and connecting development boards, routers, and test setups.                                   |

**Total Estimated Cost: LKR 53,500**

## 10.2. Budget Justification

The budget proposed is carefully planned to ensure the implementation (cost will not be too high compared to the Sri Lankan academic environment).

- **Hardware Boards:** aside from the affordability of the ESP32/STM32 boards, they are powerful enough to simulate actual IoT devices so that OTA updates can be tested. Each of the members will need its own independent development and test boards.
- **Crypto Modules:** ATECC608A secure elements have been included to represent a small set of the crypto modules; this allows hardware-based cryptography to be realistically illustrated without excess units.
- **Networking:** A common Wi-Fi router would provide a shared testbed on which all OTA update experiments would be executed in conditions that reflect practical IoT deployment situations.
- **Software Tools:** Development, and cryptographic libraries are free/open-source and thus, costs are reduced to a minimum.
- **Cloud Services:** Student plans and free options available with compute services will be used to simulate an OTA delivery environment free of subscription cost.
- **Printing & Miscellaneous:** Required in the creation of printable reports or documentation as well as smaller project installation requirements.



## References

- [1] E. D, "The Internet of Things: How the Next Evolution of the Internet Is Changing Everything," April 2011. [Online]. Available: [http://www.cisco.com/web/about/ac79/docs/innov/IoT\\_IBSG\\_0411FINAL.pdf](http://www.cisco.com/web/about/ac79/docs/innov/IoT_IBSG_0411FINAL.pdf).
- [2] Luigi Atzori , Antonio Iera, Giacomo Morabito, "The Internet of Things: A survey," ScienceDirect, 2010.
- [3] "Universal Firmware Upgrade Over-The-Air for IoT," IEEE.
- [4] Q. M. N. A. T. Meriem Bettayeb, "Firmware Update Attacks and Security for IoT Devices: Survey," [Online]. Available: [https://www.researchgate.net/publication/335089415\\_Firmware\\_Update\\_Attacks\\_and\\_Security\\_for\\_IoT\\_Devices\\_Survey](https://www.researchgate.net/publication/335089415_Firmware_Update_Attacks_and_Security_for_IoT_Devices_Survey).
- [5] "Firmware Security: Key Challenges and 11 Critical Best Practices," [Online]. Available: <https://sternumiot.com/iot-blog/firmware-security-key-challenges-and-11-critical-best-practices/>.
- [6] E. Himes, "Over-the-air Updates Using IoT: What Are They and How Do They Work?," [Online]. Available: <https://www.ptc.com/en/blogs/iiot/iot-over-the-air-update>.
- [7] S. E. JAOUHARI, "Toward a Secure Firmware OTA Updates for constrained IoT devices," ResearchGate.
- [8] "Challenges for Securing Cyber Physical Systems," [Online]. Available: <https://ptolemy.berkeley.edu/projects/chess/pubs/601/cps-security-challenges.pdf>.
- [9] R. Johnson, Designing Secure and Scalable IoT Systems.
- [10] "Over-the-Air Software Updates for IoT Devices," [Online]. Available: <https://mender.io/>.
- [11] "Eclipse Hawkbit," [Online]. Available: <https://eclipse.dev/hawkbit/>.
- [12] D. S. e. al, "Firmware Attacks: The Silent Threat to Your IoT," ResearchGate.
- [13] T. S. e. al, "A comprehensive survey on IoT attacks: Taxonomy, detection mechanisms and challenges," ScienceDirect.
- [14] D. G. H. e. al, "Machine Learning Algorithms for Anomaly Detection in IoT," Migration Letters.

- [15] S. C. e. al, "Anomaly Detection in Industrial Machinery using IoT Devices and Machine Learning: a Systematic Mapping," ResearchGate.
- [16] M. B. Chetan Bulla, "Anomaly Detection in Industrial IoT Applications Using Deep Learning Approach," ResearchGate.
- [17] A. A. M. e. al, "Industrial IoT-Based Energy Monitoring System: Using Data Processing at Edge," ResearchGate.
- [18] B. S. A. Ayan Chatterjee, "IoT anomaly detection methods and applications: A survey," Science Direct.
- [19] "Balena," [Online]. Available: <https://www.balena.io/>.
- [20] "OTA IoT Trust Framework," [Online]. Available: <https://www.internetsociety.org/iot/trust-framework/>.
- [21] "AWS IoT Device Defender," [Online]. Available: <https://aws.amazon.com/iot-device-defender/>.
- [22] "Azure Security Center for IoT," [Online]. Available: <https://azure.microsoft.com/en-us/blog/announcing-general-availability-for-the-azure-security-center-for-iot/?msocid=0c61f570f3b863310c53e348f28d62f9>.